Capstone Project Report – Real-Time Fleet Monitoring Platform

Alexis PERRIN, Malik LAHRICHI, Matthieu SIRIER

November 2026

1 Introduction & Architecture

The fleet monitoring platform processes telemetry streams from 40 delivery vehicles spread across five French regions. The architecture relies on three core components: **Apache Kafka** for message transport, **Apache Spark** (Structured Streaming and Batch) for real-time processing and aggregations, and **Apache Airflow** for batch task orchestration and monitoring.

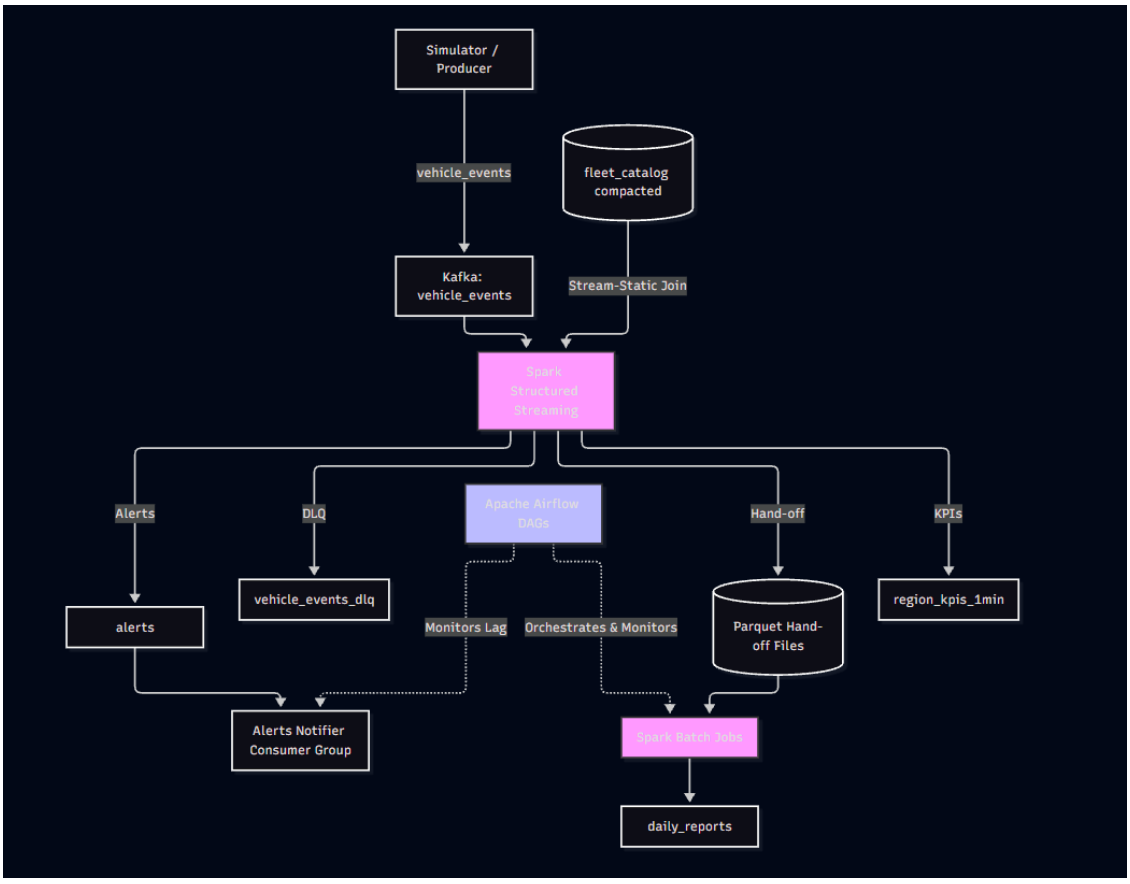


Figure 1: Global system architecture diagram.

2 Distributed-Systems Analysis

In accordance with distributed systems concepts (Chapter 1):

- **Consensus:** Handled by **KRaft** (Kafka Raft Metadata mode) for Kafka cluster metadata management without ZooKeeper, ensuring leader election and offset replication. In our setup there is a single broker with `replication.factor=1`, so there is nothing to agree on in practice: the consensus protocol is the one the platform would rely on with several brokers.
- **CAP / PACELC:** The Kafka cluster and Spark favor consistency and availability based on configured tradeoffs (`acks=all` for the producer ensures durability against network partitions, leaning towards PC/EC or PA/EL depending on tolerated latency). Spark checkpoints give at-least-once reading from Kafka: a batch that was not committed is replayed after a crash. The results are only effectively exactly-once because the sinks are idempotent (Parquet batch folders are overwritten, alerts carry a deterministic `alert_id`, and duplicate `event_id` are dropped within the watermark).

3 Part A – Kafka Layer

3.1 A.1 Log Compaction Guarantees

- **Guaranteed:** For each key, the latest value is never deleted. Replaying the topic from the beginning retrieves at least the final state of each vehicle. Offsets are preserved with gaps, and a `NULL` payload (tombstone) explicitly marks a deletion.
- **Not Guaranteed:** Immediate removal of old values. Compaction runs asynchronously only on closed segments and is triggered by `min.cleanable.dirty.ratio`, optionally delayed by `min.compaction.lag.ms`. Consequently, consumers may temporarily observe multiple versions of the same key. Tombstones are deleted only after `delete.retention.ms`.

3.2 A.2 Partition Assignment (Clean & Dirty Kill Scenarios)

Tests with two notifier instances (`A` and `B`) using the `cooperative-sticky` strategy demonstrated the rebalancing behavior:

- **Normal Scaling / Graceful Stop:** When Instance `B` starts, Instance `A` revokes partition 0 (`+[1, 2]`), which is then cleanly reassigned to `B` (`+[0]`). When `B` shuts down cleanly, partition 0 is reassigned back to `A` within seconds.
- **Dirty Kill:** When Instance `B` is abruptly killed (PID 28064), the group coordinator detects the missing heartbeat. Rebalance completes after the session timeout (46 seconds), reassigning partition 0 back to `A`.

3.3 A.3 Partition Count and Key Design

Topic	Partitions	Key	Justification
`vehicle_events`	6	`vehicle_id`	Preserves per-vehicle event ordering. Allows up to 6 parallel Spark tasks for 40 vehicles.
`fleet_catalog`	3	`vehicle_id`	Required for compaction (latest state per key). 3 partitions suffice for 40 records.
`alerts`	3	`vehicle_id`	Keeps alerts per vehicle ordered while allowing parallel notification handling.
`region_kpis_1min`	3	`region`	Preserves order for repeated window updates per region.
`vehicle_events_dlq`	3	Optional	Unparseable/corrupted messages lack strict ordering requirements.
`daily_reports`	1	Report date	One daily report; single partition guarantees global ordering without parallelism.

Table 1: Kafka Topic Configuration

3.3.1 Broker Measurements & Keying Impact

- **Key Hash Distribution:** A 15-second benchmark (321 messages across 40 vehicles) over 6 partitions resulted in an uneven distribution (4 to 8 vehicles and 26 to 74 messages per partition). Key hashing over few partitions naturally creates slight skew across Spark read tasks.
- **Random vs. Deterministic Keying:** Using a random key scatters a single vehicle's events across all partitions. With `key = vehicle_id`, all messages for that vehicle reliably route to the same partition.
- **Random Keying Impact:** Breaks ordering guarantees for standard consumers. Spark is less affected since windowed aggregations repartition data by event time anyway. However, routing by vehicle key introduces skew during heavy traffic spikes (`--hot-vehicle`).
- **Single Partition Limit:** Restricts processing to a single active consumer per group, bottlenecking throughput and disabling parallelism.
- **Broker Setup Limits:** Single-broker setup with `replication.factor=1` provides zero fault tolerance against broker failure.

3.4 A.4 Delivery Semantics per Hop

Hop	Guarantee	Mechanism & Limitations
Producer → Broker	At-least-once	Idempotent producer (`acks=all`) deduplicates retries within a session. Resets PID upon restart, requiring `run_id` in `event_id`.
Broker → Spark	At-least-once	Spark tracks Kafka offsets in its checkpoint directory and replays unfinished batches after crashes.
Spark → Alerts	At-least-once	Crash during batch output triggers replay, potentially duplicating alerts. Deduplication relies on deterministic `alert_id` (`<event_id>:point_alert`).
Alerts → Notifier	At-least-once	Manual commit after handling. Requires idempotent notification handlers to prevent duplicate actions on crash.

Table 2: End-to-End Delivery Guarantees

4 Part B – Spark Structured Streaming

4.1 B.1 Validation and Dead-Letter Queue (DLQ)

Parsing is performed using an explicit schema. Every invalid message (broken JSON, missing field, unparsable timestamp, speed outside 0-250 km/h) is systematically routed to `vehicle_events_dlq` with an explicit reason column. Special care is taken regarding SQL `NULL` semantics to prevent silent message drops during filtering operations.

4.2 State Management and Joins

- **Bounded State (B.2):** Spark drops tracked `event_id` entries once the watermark exceeds their `event_time`. Duplicate events arriving after more than 2 minutes are not recognized, trading absolute exactness for memory efficiency.
- **Stream-Static Join (B.3):** Implemented as a left outer broadcast join to retain all streaming events. Unmatched vehicles are assigned `UNKNOWN` via `coalesce`. The catalog is currently loaded as a static snapshot at startup.
- **Query 1 Idempotency (B.4):** Writes Parquet files using `overwrite` mode with `partitionOverwriteMode=dynamic`. Replayed batches overwrite their specific subfolders (`event_date=.../batch_id=...`). Point alerts include a unique `alert_id` to handle downstream duplicates.

- **Checkpointing Mechanics (B.7):** Each query maintains its own checkpoint directory tracking read offsets, committed batches, and operator states. If a checkpoint is deleted, the query restarts from scratch (skipping stopped intervals if configured with `startingOffsets=latest`) and loses internal state.

5 Part C – Spark Batch Processing

5.1 Quality Gate Thresholds

- `MIN_ROWS = 1000`: Ensures the daily pipeline processed a meaningful volume of data.
- `MAX_DUPLICATE_PCT = 0.5%`: Captures excessive duplicates while allowing small margins (simulator sends ∼1% duplicates).
- `MAX_UNKNOWN_PCT = 2.0%`: Flags missing metadata in the catalog or broken join conditions.

5.2 C.2 Verification of Partition Pruning

To answer the requirement of showing that only the partitions of the requested date are read, we rely on the physical directory layout generated by the streaming job's partition strategy (`partitionBy("event_date", "batch_id")`).

```
Rapport écrit dans /Users/alexisperrin/Desktop/pythonProject/BigDataFrameworksCapestoneProject/data/reports/report_2026-11-05.json (0 événements)
(fleet) alexisperrin@device-13 BigDataFrameworksCapestoneProject %
```

(a) Partition for 2026-11-05 (Pruned / Skipped)

```
Rapport écrit dans /Users/alexisperrin/Desktop/pythonProject/BigDataFrameworksCapestoneProject/data/reports/report_2026-11-06.json (3687 événements)
Lecture Parquet : {'number of files read': '1,080', 'size of files read': '3.8 MiB', 'number of partitions read': '186'}  (dossiers event_date/batch_id existants : 186)
(fleet) alexisperrin@device-13 BigDataFrameworksCapestoneProject %
```

(b) Partition for 2026-11-06 (Queried / Read)

Figure 2: File-system directory structure showing isolated date partitions.

When executing a daily report query filtered by date (e.g., `event_date = '2026-11-06'`), Spark applies **partition pruning**. As demonstrated by the directory tree above, Spark inspects the partition keys in the path, entirely skips scanning the unrequested folder for yesterday (Figure 2a), and restricts I/O operations exclusively to the targeted date folder (Figure 2b).

5.3 C.3 Data Skew Management

Execution with the `--hot-vehicle` option generates a marked imbalance (key `V0001` representing 41.2% of the traffic), visible in the terminal (Figure 3).

```
387780 lignes ; clé la plus fréquente : V0001 (41.2 %)
baseline          0.21 s  {'stage_id': 23, 'tasks': 8, 'task_ms_median': 43.0, 'task_ms_max': 80.0, 'shuffle_read_kb_median': 347, 'shuffle_read_kb_max': 2129}
baseline+AQE      0.13 s  {'stage_id': 28, 'tasks': 3, 'task_ms_median': 30.0, 'task_ms_max': 46.0, 'shuffle_read_kb_median': 1258, 'shuffle_read_kb_max': 2129}
salting           0.65 s  {'stage_id': 31, 'tasks': 8, 'task_ms_median': 43.0, 'task_ms_max': 45.0, 'shuffle_read_kb_median': 806, 'shuffle_read_kb_max': 896}
salting+AQE       0.23 s  {'stage_id': 37, 'tasks': 3, 'task_ms_median': 37.0, 'task_ms_max': 48.0, 'shuffle_read_kb_median': 1484, 'shuffle_read_kb_max': 2065}
Les 4 variantes donnent exactement le même résultat.
Résultats écrits dans /Users/alexisperrin/Desktop/pythonProject/BigDataFrameworksCapestoneProject/data/reports/skew_study_2026-11-06.json
Spark UI : http://device-13.home:4040 - Entrée pour quitter...
```

Figure 3: Comparative results of the four optimization variants.

In the Spark UI (Figure 4), the task processing the hot key experiences a maximum execution time of **80.0 ms** (compared to a median of **43.0 ms**), causing a straggler effect that delays the completion of the stage.

Analysis:

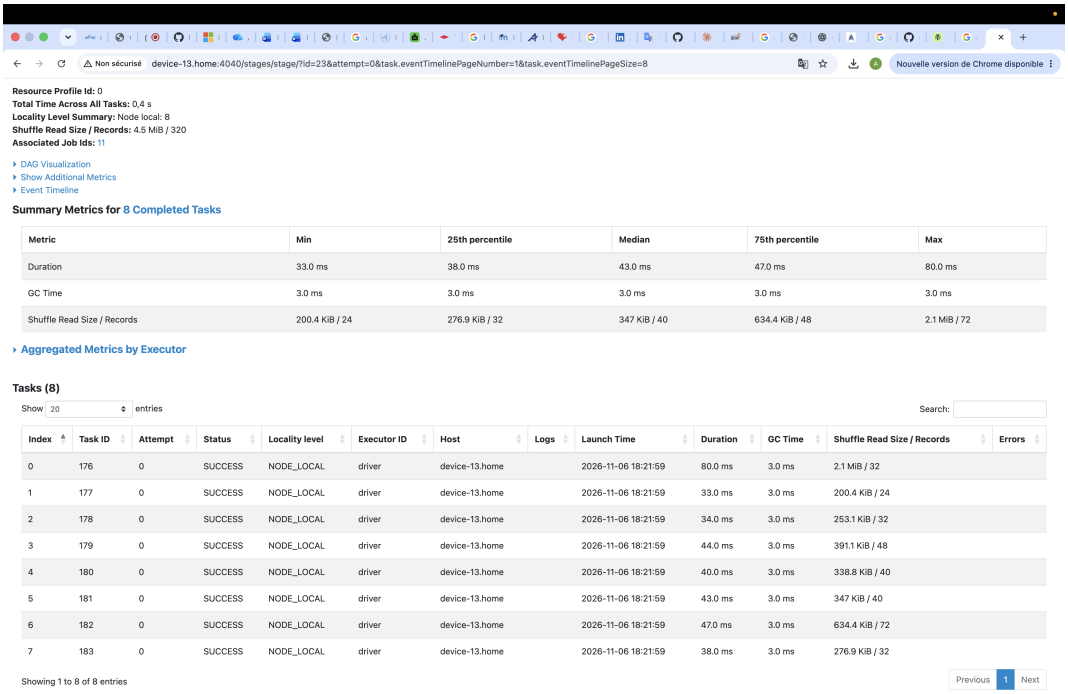


Figure 4: Task distribution and shuffle metrics in the Spark UI.

- **Baseline + AQE:** Reduces execution time to 0.13 s thanks to dynamic shuffle partition merging.
- **Salting:** Effectively distributes the load (maximum shuffle reduced to 896 KiB) but introduces a computational overhead (0.65 s).
- **Salting + AQE:** Provides a balanced compromise (0.23 s) by combining salt partitioning with AQE dynamic optimizations.

6 Part D – Airflow Orchestration

6.1 DAG Architecture

- `fleet_daily_pipeline` (`@daily`): Waits for Parquet data, runs data quality checks, branches based on quality verdict, and publishes either the daily report or a quality alert.
- `kafka_lag_monitor` (Every 5 min): Monitors consumer lag on the `alerts-notifier` group and triggers a `consumer_lag_high` alert if lag exceeds `LAG_THRESHOLD` (10 messages).

6.2 Execution Mechanics and Idempotence

- **Logical Date vs. Execution Time:** Runs evaluate data based on logical date (`ds`). A run for `2026-10-19` processes that specific date regardless of when the execution actually occurs.
- **Backfill & Catchup:** Setting `catchup=False` prevents uncontrolled execution backlogs on startup. A 7-day backfill (`2026-10-12` to `2026-10-18`) executed deterministically without duplicate side-effects.
- **Application-Level Idempotence:** Kafka's producer idempotence only covers intra-session retries. Business-level idempotence is enforced via deterministic keys (`<date>:data_quality_failed`) and pre-publication topic checks.
- **Sensor Optimization:** The `wait_for_files` sensor uses `reschedule` mode to release worker slots between checks, preventing task starvation under the single-threaded `SequentialExecutor`.

7 Part E – Experiments and Performance Study

7.1 Method

For each experiment the hypothesis was written before the run. The raw outputs (counts, gap checks, logs, Spark UI screenshot) are in `report/screenshots/` of the project. Output integrity is checked with two scripts: `scripts/count_outputs.sh` (rows and distinct `event_id` in Parquet, alerts, DLQ messages) and `scripts/check_gaps.py` (sequence numbers of a simulator run that are missing from the Parquet files).

Reference level: even without any failure, about 3% of the sequence numbers are missing from Parquet, because 1% of the messages are corrupted (they go to the DLQ) and 2% are late and dropped by the watermark. A failure that really loses data has to show more than that, or a long contiguous hole.

7.2 Failure Experiments

- **E.1 – `kill -9` on the streaming job, then restart.** *Hypothesis:* the checkpoint lets Spark resume from the last committed offsets, with no loss and no duplicates in the output. *Observation:* after the restart the Parquet files contain no duplicated `event_id`, and the missing sequence numbers stay at the reference level, without any long hole. This matches the design: a replayed batch overwrites its own `batch_id` folder.

- **E.2 – Notifier stopped for about 10 minutes.** *Hypothesis:* the lag of the notifier group grows, the `kafka_lag_monitor` DAG raises an alert, and everything clears once the notifier is back. *Observation:* the lag was 0 on the three partitions before the stop. At the 20:25 UTC run the DAG read 106, 106 and 75 messages and published one `consumer_lag_high` alert per partition. After the restart the lag went back to 0 and the 20:30 UTC run published nothing. The monitor detects a stopped consumer without touching it.

- **E.3 – Kafka broker stopped for 1 minute** (21:37:34 to 21:38:36), with the simulator at 20 events/s, the streaming job and the notifier running. *Hypothesis:* the producer queues and retries, the Spark job survives or fails when its timeouts are exceeded, and the system recovers without loss because the broker has its data on disk. *Observation:* the producer kept the messages in its local queue and delivered all of them when the broker came back (3300 delivered, 0 failed, 0 left in the queue). This only holds while the producer stays alive: in a first attempt we stopped the simulator during the outage and the 343 messages still in its in-memory queue were lost. The Spark job did *not* survive: when we checked, the process no longer existed, and its log ended with connection errors to the broker followed by a py4j error (`Py4JNetworkException`, `Received empty command`); we did not capture the first exception of the trace, so we cannot name the root cause. The same happened in the first attempt. After restarting the job with the same checkpoint, it caught up: the sequence reached 3280, with 87 numbers missing (2.7%, longest hole of 2), 41 926 rows for 41 926 distinct `event_id` and 1 278 alerts all with a distinct `alert_id`. This is the reference level, so the outage itself lost nothing. With one broker and a replication factor of 1, the data written before the outage stays on disk, but the streaming job needs something that restarts it (a supervisor, a restart policy or an Airflow task).

- **E.4 – Burst at 10× the normal rate** (200 events/s for about 100 s). *Hypothesis:* the limit is Spark and not Kafka. *Observation:* Kafka took the burst without trouble (19 805 delivered, 0 failed). We then polled the Parquet output every 20 seconds: at the first poll (21:52:13) the sequence was already at 19 600 and did not change afterwards, so Spark had no backlog left. 560 numbers are missing (2.9%, longest hole of 2), the reference level, and the difference between the 19 805 messages delivered and the 19 600 sequence numbers is the 1% of duplicates sent by the simulator. Kafka was clearly not the limit, but Spark was not either at this load. We did not measure the backlog during the burst, only after it, so we give no latency. The Spark UI shows a processing rate of 233 to 253 events/s per query, with the four queries in the same local application: we read this as the order of magnitude of the ceiling of this setup, which is an inference, not a test.

7.3 Performance Study

We did not have the time to compare several configurations properly, so we do not give a comparison table. What we measured: the job handles 200 events/s without a lasting backlog, and the Spark UI processing rate is about 250 events/s per query on our machine (local mode, four queries, 6 shuffle partitions, at most 5 000 offsets per micro-batch). We did not measure the latency between the event time and the alert, nor the effect of the number of shuffle partitions, the trigger interval or `maxOffsetsPerTrigger`.
Recommendation: for this project (20 events/s, bursts of 200) the current configuration is enough. If the input had to go clearly above 250 events/s, those three settings are where we would start, one at a time, watching the processing rate in the Spark UI; this has not been tested.

8 Individual Contributions

- **Alexis Perrin:** Spark code of parts B and C (streaming job, quality and report jobs, skew study), questions A.1 and A.2.
- **Malik Lahrichi:** Airflow part (part D): daily pipeline DAG with sensor, branching and retries, idempotent publication to Kafka, consumer-lag monitoring DAG, 7-day backfill check. Questions A.3 and A.4. All the failure experiments of part E, with the helper scripts `scripts/count_outputs.sh` and `scripts/check_gaps.py`, and the raw outputs in `report/screenshots/`.
- **Matthieu Sirier:** A.2 Clean & Dirty Kill Scenarios, Repository and coordination, this report.